LESSON 06 · DOCKER · 50 MIN

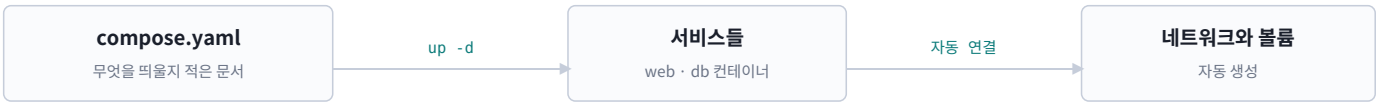
6교시 Docker Compose — 여러 컨테이너를 한 번에

앱 하나에 DB 하나만 붙어도 명령어가 길어지고 순서가 꼬입니다. Compose는 그 전부를 파일 하나로 적어 두고 명령 한 줄로 띄웁니다. 설정의 각 줄이 어떤 구조를 만드는지 확인하며 진행합니다.

개념 12분 · 실습 25분 · 마무리 정리 13분

OVERVIEW

compose 파일 하나가 만드는 것



> 같은 compose 안의 컨테이너들은 **서비스 이름으로 서로를 부릅니다.** web에서 DB 주소를 localhost로 적으면 절대 연결되지 않습니다. 각 컨테이너의 localhost는 자기 자신이기 때문입니다.

REFERENCE

이 시간에 쓰는 명령

명령	하는 일	언제 쓰나
`services`	띄울 컨테이너들의 목록	파일의 뼈대
`build: . / image: ...`	직접 빌드 / 기존 이미지 사용	내 앱은 build, DB는 image
`ports`	호스트:컨테이너 포트 개방	밖에서 접속해야 하는 서비스만
`environment`	환경변수 전달	비밀번호, 접속 주소 등
`volumes`	데이터 보존	DB에는 사실상 필수
`depends_on`	시작 순서 지정	DB를 먼저 띄울 때
`docker compose up -d / down`	전체 시작 / 정리	매일 쓰는 두 명령
`docker compose logs -f <서비스>`	서비스 로그 보기	안 될 때 가장 먼저
`docker compose exec <서비스> sh`	컨테이너 안으로	내부 확인

FILES

실습용 파일

compose.yaml

```
services:
  web:
    build: .
    ports:
      - "8080:3000"
    environment:
      DB_HOST: db
    depends_on:
      - db

  db:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD: secret
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```

HANDS-ON

실습 순서

STEP 01 compose 파일 작성

5교시 폴더에 compose.yaml 을 만듭니다.

```
cd docker-app
# 아래 '실습용 파일'의 compose.yaml 을 이 폴더에 만듭니다
ls
```

확인 포인트

- [ ] Dockerfile, server.js, package.json, compose.yaml 이 한 폴더에 있다

> YAML은 **들여쓰기가 문법**입니다. 반드시 스페이스 2칸을 쓰고, 탭은 절대 쓰지 마세요. 예전 자료에 나오는 **version:** 줄은 이제 필요 없습니다.

STEP 02 한 번에 띄우기

명령 한 줄로 컨테이너 두 개를 실행합니다.

```
docker compose up -d
docker compose ps
docker network ls
docker volume ls
docker compose logs db
# 브라우저에서 http://localhost:8080 접속
```

확인 포인트

- [ ] web과 db 두 서비스가 모두 running 이다
- [ ] 네트워크와 볼륨이 자동으로 만들어져 있다
- [ ] db 로그에 database system is ready to accept connections 가 보인다

> 처음에는 postgres 이미지를 받느라 시간이 걸립니다. 로그가 멈춘 것처럼 보여도 기다리세요.

STEP 03 서비스 이름으로 통신 확인 — 오늘의 핵심

컨테이너끼리 어떻게 서로를 찾는지 확인합니다.

```
docker compose exec web sh
# 컨테이너 안에서:
getent hosts db
ping -c 2 db
exit
```

확인 포인트

- [ ] db 의 내부 IP 주소가 출력된다
- [ ] ping 이 응답한다

> **db 라는 이름이 곧 주소입니다.** compose가 만든 네트워크 안에서 서비스 이름이 호스트 이름 역할을 합니다. 그래서 앱 설정에 DB_HOST=db 라고 적은 것입니다. localhost로 적으면 web 컨테이너 자기 자신을 가리켜 연결에 실패합니다.

STEP 04 down 해도 데이터가 남는지 확인

볼륨의 효과를 직접 검증합니다.

```
docker compose exec db psql -U postgres -c "CREATE TABLE t(id int);"
docker compose exec db psql -U postgres -c "INSERT INTO t VALUES (1);"
docker compose exec db psql -U postgres -c "SELECT * FROM t;"
docker compose down
docker compose up -d
docker compose exec db psql -U postgres -c "SELECT * FROM t;"
```

확인 포인트

- [ ] down 후 docker compose ps 결과가 비어 있다
- [ ] 다시 up 하고 조회하면 데이터 1이 그대로 남아 있다

> **down은 컨테이너와 네트워크를 지우지만 볼륨은 남깁니다.** 그래서 개발 중 몇 번을 껐다 켜도 DB 데이터가 유지됩니다.

STEP 05 down -v 의 차이 확인

볼륨까지 지우면 무엇이 사라지는지 확인합니다.

```
docker compose down -v
docker volume ls
docker compose up -d
docker compose exec db psql -U postgres -c "SELECT * FROM t;"
```

확인 포인트

- [ ] volume ls 에서 pgdata 가 사라졌다
- [ ] 다시 조회하면 relation "t" does not exist 오류가 난다

> **-v 는 되돌릴 수 없습니다.** 개발 DB를 초기화할 때는 유용하지만, 습관적으로 붙이면 사고가 납니다.

STEP 06 멀티스테이지 빌드로 이미지 줄이기

빌드에만 필요한 것을 최종 이미지에서 빼 봅니다.

```
# Dockerfile 을 아래처럼 두 단계로 수정
FROM node:20 AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install

FROM node:20-alpine
WORKDIR /app
COPY --from=builder /app/node_modules ./node_modules
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

확인 포인트

- [ ] docker build -t myapp:slim . 로 빌드한다
- [ ] docker images 로 myapp:v4 와 myapp:slim 의 SIZE를 비교한다
- [ ] v4 _____ MB / slim _____ MB

> 첫 빌드에서 node:20 이미지(약 1GB)를 새로 받으므로 시간이 걸립니다. 미리 받아두면 수업이 매끄럽습니다.
> **COPY --from=builder** 는 앞 단계의 결과물만 가져옵니다. 빌드에 쓰인 무거운 도구들은 최종 이미지에 실리지 않습니다.

STEP 07 일부러 고장 내고 고치기

문제가 생겼을 때 확인하는 순서를 몸에 익힙니다.

```
# compose.yaml 의 ports 를 "8080:9999" 로 잘못 바꾸고 저장
docker compose up -d
# 브라우저 접속 → 실패
docker compose ps
docker compose logs web
# ports 를 "8080:3000" 으로 되돌리고
docker compose up -d
```

확인 포인트

- [ ] ps 의 PORTS 칸에서 잘못된 매핑을 눈으로 확인했다
- [ ] up -d 를 다시 하면 바뀐 서비스만 재생성된다 (Recreated 메시지)

> 확인 순서를 외우세요. **compose ps로 상태 → logs로 이유 → 설정 수정 → up -d**. 이 순서면 대부분의 문제가 5분 안에 잡힙니다.

STEP 08 마무리 정리

오늘 만든 자원을 정리합니다.

```
docker compose down -v
docker ps -a
docker system df
```

확인 포인트

- [ ] 컨테이너와 볼륨이 정리되었다

TROUBLESHOOTING

자주 만나는 오류

증상	원인	해결
`port is already allocated`	호스트 포트 충돌	ports 의 왼쪽 숫자를 바꾸거나 기존 컨테이너 중지
web에서 `DB` 연결 실패	주소를 localhost로 적었거나 DB가 아직 준비 전	호스트를 서비스 이름(db)으로 · 앱에 재시도 로직 추가
`yaml: line N: did not find expected key`	들여쓰기 오류 (탭 사용)	스페이스 2칸으로 통일 · 편집기에서 탭 표시 켜기
`depends_on` 했는데도 `DB` 연결 실패	depends_on은 시작 순서만 보장	DB가 접속을 받을 준비까지 기다리지 않음 → 앱에서 재시도
`down` 했더니 데이터가 사라짐	volumes 미설정 또는 -v 옵션 사용	compose.yaml 의 volumes 확인 · -v 는 신중히

REVIEW

복습 체크리스트 — 답할 수 있으면 통과

- ☐ 같은 compose 안의 컨테이너끼리는 서로를 무엇으로 부르나요?
- ☐ depends_on 이 보장하는 것과 보장하지 않는 것은 각각 무엇인가요?
- ☐ docker compose down 과 down -v 의 차이는 무엇인가요?
- ☐ ports 를 적지 않은 서비스는 어떻게 되나요? (db가 그렇습니다)
- ☐ 멀티스테이지 빌드가 이미지를 줄이는 원리는 무엇인가요?

한 줄 정리

Compose는 새로운 기술이 아니라 지금까지 친 명령어들을 파일로 적어 둔 것입니다. 팀원에게 환경 설명서를 주는 대신 이 파일 하나를 주면 됩니다.

Git	파일의 이동(add/commit)과 되돌리기(restore/reset), 갈래(branch/merge), 그리고 원격 동기화(push/fetch/pull)
Docker	이미지와 컨테이너의 구분, 포트 매핑, Dockerfile의 레이어 캐시, 볼륨, 그리고 Compose
이어서 공부하면 좋은 것	개발용 bind mount로 코드 즉시 반영 · 헬스체크 · 이미지 경량화 · 컨테이너 오케스트레이션 기초

WORKSHEET

막힌 STEP 번호, 화면에 나온 오류 메시지, 강사에게 물어볼 것을 적어 두세요.